

Grouped-Query Attention & KV-Cache.

At generation time the dominant memory cost is the KV-cache: every past token's keys and values, stored per head, per layer. Multi-head attention keeps a full K/V head for every query head; multi-query attention shares *one*; grouped-query attention interpolates with G groups. We derive the cache-size formula and evaluate it for a 32-head, 32-layer model at context 4096, showing the exact $4\times$ saving of GQA-8 and $32\times$ of MQA. Every number below is computed in full — nothing summarised or deferred.

THE EQUATION

$$\text{cache} = 2 \cdot L \cdot T \cdot (n_{kv} d_{\text{head}}) \cdot b_{\text{bytes}} \quad \text{reduction} = \frac{H}{n_{kv}} = \frac{H}{G}$$

Worked example. $H = 32$ query heads, $d_{\text{head}} = 128$, $L = 32$ layers, fp16 (2 bytes), $T = 4096$. MHA ($n_{kv}=32$), GQA-8 ($n_{kv}=8$), MQA ($n_{kv}=1$): 2.0, 0.5, 0.0625 GiB.

Topic 2 of 3 in this module · companion to the course page · v1.0

Contents

1	The claim: cache scales with KV heads, not query heads	2
2	A diagram: how heads share K/V	3
3	The derivation: where the formula comes from	3
4	Worked example: $H = 32$, $d_{\text{head}} = 128$, $L = 32$, $T = 4096$, fp16	3
5	Properties / consequences that matter	4
6	Where this sits in the track	4
A	Reproducible (NumPy)	5

1 The claim: cache scales with KV heads, not query heads

Standard *multi-head attention* (MHA) gives every one of the H query heads its own key and value projection: H query heads, H key heads, H value heads. *Multi-query attention* (MQA) keeps all H query heads but lets them share a *single* key head and a single value head. *Grouped-query attention* (GQA) is the middle ground: G key/value groups, each shared by H/G query heads. Let n_{kv} be the number of key/value heads ($n_{kv} = H$ for MHA, G for GQA, 1 for MQA). The autoregressive KV-cache stores K and V for every layer and every cached token:

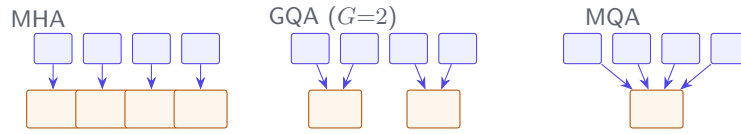
$$\text{cache} = 2 \cdot L \cdot T \cdot (n_{kv} d_{\text{head}}) \cdot b_{\text{bytes}} \quad (1)$$

The leading 2 counts K and V . Crucially the cache depends on n_{kv} , *not* on the number of query heads H — so cutting KV heads cuts memory in direct proportion.

Symbol	Meaning	Value / shape
H	query heads	32
n_{kv}	key/value heads	H (MHA), G (GQA), 1 (MQA)
G	GQA groups	here 8
d_{head}	per-head dimension	128
L	transformer layers	32
T	cached sequence length	4096
b_{bytes}	bytes per scalar	2 (fp16)

Query heads are cheap to keep — they are recomputed for the single current token each step. Keys and values are expensive because they *accumulate*: every token ever generated leaves a K and V behind, forever, in every layer. GQA’s insight is that many query heads can attend to the *same* keys and values without much loss, so you pay to store K/V once per group instead of once per head. You shrink the part that grows with sequence length, and leave the cheap query heads untouched.

2 A diagram: how heads share K/V



Blue = query heads (always 4 here); amber = stored K/V heads. MHA stores 4, GQA-2 stores 2, MQA stores 1. The cache shrinks with the amber count.

3 The derivation: where the formula comes from

Per layer, the cache holds the keys for all T tokens and all n_{kv} KV heads, each a d_{head} -vector, plus an identical block for values:

$$\text{per layer} = \underbrace{2}_{K,V} \times T \times n_{kv} \times d_{\text{head}} \text{ scalars.}$$

Multiply by L layers and b_{bytes} bytes per scalar to get Eq. (1). The ratio of any two schemes is just the ratio of their KV-head counts, since $L, T, d_{\text{head}}, b_{\text{bytes}}$ are shared:

$$\frac{\text{cache}_{\text{MHA}}}{\text{cache}_{\text{GQA}}} = \frac{H}{G}, \quad \frac{\text{cache}_{\text{MHA}}}{\text{cache}_{\text{MQA}}} = \frac{H}{1} = H \quad (2)$$

With $H = 32$ and $G = 8$, each group is shared by $H/G = 32/8 = 4$ query heads, and the cache shrinks by $32/8 = 4\times$. MQA shrinks it by the full $32\times$.

4 Worked example: $H = 32$, $d_{\text{head}} = 128$, $L = 32$, $T = 4096$, **fp16**

First compute the common factor $2 \cdot L \cdot T \cdot d_{\text{head}} \cdot b_{\text{bytes}}$ that multiplies n_{kv} :

$$2 \cdot 32 \cdot 4096 \cdot 128 \cdot 2 = 2 \cdot 32 \cdot 4096 \cdot 256 = 67,108,864 \text{ bytes per KV head} = \frac{1}{16} \text{ GiB.}$$

So the cache is exactly $n_{kv}/16$ GiB. Evaluate the three regimes:

$$\text{MHA: } n_{kv} = 32 : 32 \times 67,108,864 = 2,147,483,648 \text{ B} = 2.0000 \text{ GiB,}$$

$$\text{GQA-8: } n_{kv} = 8 : 8 \times 67,108,864 = 536,870,912 \text{ B} = 0.5000 \text{ GiB,}$$

$$\text{MQA: } n_{kv} = 1 : 1 \times 67,108,864 = 67,108,864 \text{ B} = 0.0625 \text{ GiB.}$$

GQA-8 is exactly $2.0/0.5 = 4\times$ smaller than MHA; MQA is $2.0/0.0625 = 32\times$ smaller. The full table, with quality/speed notes:

Scheme	n_{kv}	cache (GiB)	reduction	trade-off
MHA	32	2.0000	$1\times$	full quality, largest cache
GQA-8	8	0.5000	$4\times$	$\approx$ MHA quality, $4\times$ less memory
GQA-4	4	0.2500	$8\times$	small quality cost
MQA	1	0.0625	$32\times$	fastest, measurable quality drop

The heatmap shades cache footprint relative to MHA (intensity $\propto$ size, capped at indigo!80 for the largest):

	MHA (32)	GQA-8 (8)	GQA-4 (4)	MQA (1)
cache GiB	2.0000	0.5000	0.2500	0.0625

(Shading is linear in cache size: $2.0 \rightarrow !80$, $0.5 \rightarrow !20$, $0.25 \rightarrow !10$, $0.0625 \rightarrow !3$.) The per-token cost is the same picture without T : MHA stores $2 \cdot 32 \cdot 32 \cdot 128 \cdot 2 = 524,288$ bytes (0.5 MiB) of K/V *per generated token*, so the cache grows by half a megabyte each step — exactly what GQA cuts to an eighth.

5 Properties / consequences that matter

1. **Linear in T .** Cache grows linearly with context length; halving KV heads doubles the context you can fit in the same memory.
2. **Bandwidth, not just capacity.** Decoding is memory-bound: each step *reads* the entire cache. Shrinking it $4\times$ cuts the bytes moved per token $4\times$, directly speeding up generation throughput.
3. **Quality knee.** GQA with a moderate G (e.g. 8) recovers nearly all MHA quality, while MQA’s single shared head shows a measurable degradation — GQA is the chosen compromise in LLaMA-2 70B and most large open models.
4. **Query heads unchanged.** All $H = 32$ query heads remain; expressivity of the *query* side is untouched. Only the shared K/V is reduced, and at inference the group’s K/V is broadcast to its query heads.

GQA reduces the KV-cache, not the attention *compute*: each query head still attends over all T keys, so the $O(T^2)$ score computation is unchanged — the win is memory and bandwidth, not FLOPs. Also beware double-counting: the formula already includes both K and V via the leading 2; writing $4 \cdot L \cdot T \cdot n_{kv} d_{\text{head}} b$ “to be safe” overstates the cache by $2\times$. And the cache is per *sequence in the batch* — multiply by batch size B for the true footprint, which is exactly why batched serving makes the cache the binding memory constraint.

6 Where this sits in the track

Module 0.2 introduced the KV-cache and its $O(T)$ memory growth for plain MHA; this topic is the architectural fix the LLaMA family adopted. GQA pairs with RoPE (Topic 1) — rotated keys

are what gets cached — and with the efficient FFN of Topic 3 to make large decoders servable. The bandwidth-bound decoding picture here is taken further in Module 7.3 (inference systems), where the cache size set by n_{kv} determines achievable batch size and tokens-per-second.

A Reproducible (NumPy)

Inputs: $H = 32$, $d_{\text{head}} = 128$, $L = 32$, $T = 4096$, $b_{\text{bytes}} = 2$. The snippet reproduces the per-KV-head factor and the three cache sizes (2.0, 0.5, 0.0625 GiB) plus the reduction factors.

```
import numpy as np
H, d_head, L, T, b = 32, 128, 32, 4096, 2
def cache(n_kv): return 2*L*T*(n_kv*d_head)*b # bytes
for name, n_kv in [("MHA",32),("GQA-8",8),("GQA-4",4),("MQA",1)]:
    c = cache(n_kv)
    print(name, "n_kv=",n_kv, "GiB=%.4f"%(c/2**30), "reduction=", 32//n_kv)
# MHA    n_kv= 32 GiB=2.0000 reduction= 1
# GQA-8  n_kv=  8 GiB=0.5000 reduction= 4
# GQA-4  n_kv=  4 GiB=0.2500 reduction= 8
# MQA    n_kv=  1 GiB=0.0625 reduction= 32
print(cache(1)) # 67108864 bytes per KV head (=1/16 GiB)
print(2*L*(H*d_head)*b) # 524288 bytes/token, MHA (0.5 MiB)
```